RV UNIVERSITY

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

CS2201 - DATABASE MANAGEMENT SYSTEM

Mini Project Report

IT Incident Tracking System

Submitted

By

Krishna H Chandawar (1RUA25BCA0052)

Aditya C (1RUA25BCA0007)

Maanas J (1RUA25BCA0057)

Under the guidance of:

Dr./Prof. Pranjul Shukla

School of Computer Science and Engineering

RV University, Bangalore

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

CERTIFICATE

Certified that the CS2201 - Database Management System mini project titled IT Incident Tracking System is carried out by Krishna H Chandawar (1RUA25BCA0052), Aditya C (1RUA25BCA0007), and Maanas J (1RUA25BCA0057), bonafide students of the School of Computer Science and Engineering, RV University, Bengaluru, during the academic year 2026-27. The project demonstrates a database-backed process for reporting, assigning, tracking, resolving, and auditing IT incidents. It is certified that the corrections and suggestions from continuous internal evaluations have been incorporated into the project and this report.

Dr./Prof. Pranjul Shukla

Faculty Guide Program Director

REPORT CONTENTS

Section	PAGE NUMBER
Abstract	Problem, objectives, methodology, tools, and outcomes
1. Introduction	Domain, motivation, significance, and objectives
2. Scope of the Project	Boundaries, assumptions, applications, and limitations
3. Coding	Architecture, database design, implementation, and workflow
4. Screenshots	Incident interface and database-status evidence
5. Conclusion	Summary, findings, contribution, and future work
6. AI Tool Usage Report	Tool, purpose, prompt, outputs, and contribution
Appendix	Database and workflow summary

Abstract

Problem statement: Organisations need a reliable way to register IT incidents, assign responsibility, communicate progress, and retain a complete history. Manual or scattered reporting can lose important details, delay resolution, and make accountability difficult.

Objectives: The IT Incident Tracking System provides a role-based workflow in which users report incidents, technicians work on assigned tickets, managers review and control resolution, and administrators manage approvals and audit activity.

Methodology: The project combines a Java browser/console application with a MySQL relational database. JDBC and parameterised SQL are used for persistence, while role-based services, transactions, validation, password hashing, status updates, and activity logs support the incident lifecycle.

Tools used: Java, JDK, MySQL 8, JDBC, MySQL Connector/J, HTML, CSS, JavaScript, PowerShell, Maven/NetBeans project tooling, and Git.

Key outcomes: The system creates unique incident tokens, calculates priority, supports assignment and status changes, protects role-specific operations, stores activity history, and provides administration and database-status views.

This report presents the implemented scope, database design, working flow, verification approach, and limitations of the academic project.

Project profile

Item	Observed project details
Application	IT Incident Tracking System
Interface	Browser dashboard and console mode
Server	Java application with embedded HTTP server
Database	MySQL 8 accessed through JDBC
Workflow	User, technician, manager, and admin roles
Verification	Source, build, schema, and workflow inspection
Verification	Six passing unit tests; live SELECT 1 health query succeeded

1. Introduction

➔ Problem domain

IT support teams handle incidents that range from access and software issues to network, hardware, security, and database failures. Each request requires a clear description, priority, owner, current status, estimated resolution, and traceable communication. A central ticket database makes this information searchable and consistent throughout the incident lifecycle.

➔ Motivation and significance

The project was selected to apply database-management concepts to a practical service-desk scenario. It demonstrates how authentication, relational integrity, transactions, role permissions, status history, and audit records work together in one system.

The system is intended as a focused academic prototype. It is suitable for demonstrating incident submission, technical work allocation, managerial review, administrative approval, and persistent reporting without the complexity of a commercial IT service-management platform.

➔ Objectives

? Provide secure registration and login for users, technicians, managers, and administrators.

? Register incidents with requester information, issue type, description, priority, and a unique token.

? Allow technicians to claim, update, reassign, and estimate incident resolution work.

? Allow managers to review, resolve, close, and reopen incidents according to workflow rules.

? Preserve ticket updates, role-based activity logs, and administrative audit entries.

? Apply validation, prepared statements, transactions, and password hashing to important operations.

? Provide browser and console access backed by the same MySQL data model.

The intended outcome is a dependable, understandable workflow from incident submission to auditable resolution.

2. Scope of the Project

➔ Included features

The application supports account registration and login, incident creation, issue-type selection, automatic priority calculation, unique token generation, personal ticket tracking, technician work queues, status updates, assignment and reassignment, estimated resolution time, manager

review, closure, reopening, activity history, administrative approvals, role management, and database-status reporting.

➔ Boundaries and limitations

? The project is designed for local academic demonstration and is not a complete enterprise ITSM deployment.

? It requires a configured MySQL server and database credentials; production hosting and high availability are outside scope.

? Notifications, attachments, SLA escalation, analytics dashboards, and external directory integration are not implemented.

? Authentication is application-managed; deployment still requires secure secrets, HTTPS, backups, and operational monitoring.

? Testing focuses on application behavior and selected database operations rather than load, disaster recovery, or penetration testing.

? Reopening and closure are workflow actions recorded by the system; they do not represent a full change-management process.

➔ Assumptions and intended applications

The setup assumes Java, MySQL 8, a reachable incident_tracking database, and a user account with the required schema privileges. The application can be used as a teaching example for relational design, JDBC programming, role-based access, ticket workflows, and auditability.

The project is intended for student demonstration, small internal support teams, and further extension into a more complete service-desk solution.

3. Coding

Development methodology

The implementation follows a layered Java design. Browser requests are handled by the embedded HTTP server and web service classes, while console mode uses the same core access and ticketing services. Validation occurs before database writes, and prepared statements are used for user-supplied values.

Transactions group related ticket and activity updates so that an incident is not recorded without its initial history entry. Role checks determine which service operations are available to USER, TECHNICIAN, MANAGER, and ADMIN accounts.

Architecture and data flow

Layer	Role
Browser/console UI	Collects credentials, incidents, updates, and review actions
Service layer	Validates requests, applies roles, calculates priority
JDBC persistence	Executes parameterised SQL and transactional writes
Relational database	Stores accounts, tickets, updates, and audit history
Administration	Approvals, role management, statistics, and audit review

A user authenticates and submits an incident through the browser or console. The service validates the issue data, computes priority, generates a token, and writes the ticket and initial update to MySQL. A technician then claims or receives the ticket and adds progress updates. Managers review the work, resolve or close the ticket, and can reopen it when further action is required. Each important transition is written to the relevant activity log.

Tools and libraries

The project uses Java, the JDK HTTP server, JDBC, MySQL Connector/J, MySQL 8, HTML, CSS, JavaScript, PowerShell scripts, and Maven/NetBeans build support. PBKDF2-HMAC-SHA256 with a per-user salt is used for password storage. UUID-based tokens identify incidents without exposing sequential ticket numbers.

Database design

The relational schema is centred on app_users and tickets. User records store identity, department, designation, role, approval state, salt, and password hash. Tickets store requester details, issue type, description, priority, status, technician assignment, estimated time, timestamps, and closure information.

Supporting tables retain ticket_updates and separate user, technician, manager, and administrator activity logs. Foreign keys, unique keys, indexes, enumerated status values, and audit records support integrity, traceability, and efficient operational queries.

Table	Purpose and key relationships
app_users	Accounts, roles, approval state, salts, and password hashes
tickets	Incident details, priority, status, assignment, and timestamps
ticket_updates	Chronological progress and status messages
user_activity_log	Requester actions and incident history
technician_activity_log	Technical work and status changes
manager_activity_log	Review, resolution, closure, and reopening
admin_audit_log	Administrative approvals, role changes, and audit actions
feedback	Customer rating and message, with rating range constraint

Database access is role-aware and uses parameterised SQL. Ticket creation and updates are committed as consistent units, while activity records provide a chronological explanation of who performed each action and when.

Coding: incident workflow and access control

The user interface provides login, registration, incident submission, ticket lists, history, and role-specific menus. Issue types include operating-system, application, account, security, network, database, hardware, VPN, performance, backup, permissions, and other support categories.

Priority is calculated from issue type, description keywords, and requester designation. Security, data-loss, and production-down conditions are treated as critical; hardware, backup, VPN, and senior-designation cases can be high priority; network and database issues are medium; remaining cases are low unless a manager overrides the value.

Technicians can claim unassigned work, update progress, add notes, set estimated resolution time, and reassign tickets. Managers can review all incidents, resolve and close them, reopen completed work, and inspect operational history. Administrators approve registrations, manage roles, and inspect audit activity.

Authentication and ticket processing

User accounts are validated before registration. Passwords are never stored as plain text: a random salt and PBKDF2-HMAC-SHA256 hash are stored instead. Technician and manager registrations remain pending until an administrator approves them, while unapproved accounts cannot authenticate.

After authentication, the account role controls the available workflow. A USER creates and tracks personal incidents; a TECHNICIAN works assigned incidents; a MANAGER controls operational review; and an ADMIN manages approvals, roles, statistics, and audit records.

Incident lifecycle

The lifecycle begins with Open, followed by Allocated, In Progress, Waiting, Pending Manager Review, and Resolved states as appropriate. A manager can close a resolved incident, while a closed incident may be reopened for additional work. Updates record the actor, role, status, estimated time, and message.

Priority and audit boundaries

Priority helps order the work queue but does not replace human judgement. The system records administrative actions and role-based activity for accountability. Database availability can be checked locally, but production monitoring, notification delivery, and service-level measurement require additional operational components.

Results, observations, and challenges

The implemented system provides an end-to-end incident workflow: registration, authentication, incident creation, priority calculation, ticket retrieval, technician processing, manager review, closure, reopening, and history display. The project also includes a web server mode and a console mode for local operation.

Verification	Observed result
Project structure	Dedicated access, incident, ticketing, web, and database packages
Database design	Relational tables, keys, indexes, statuses, and audit records
Authentication	Salted PBKDF2 password hashing and approval-aware login
Workflow	Token-based incident lifecycle with role-specific operations
Scope	Academic local prototype; production controls remain future work

Verification was performed through source review, compilation/build artefacts, database scripts, and inspection of the implemented workflow. The project structure contains dedicated access, incident, ticketing, web, department, and statistics packages. Verification confirms the intended database-backed design; a production deployment would require broader integration and security testing.

Challenges and effectiveness

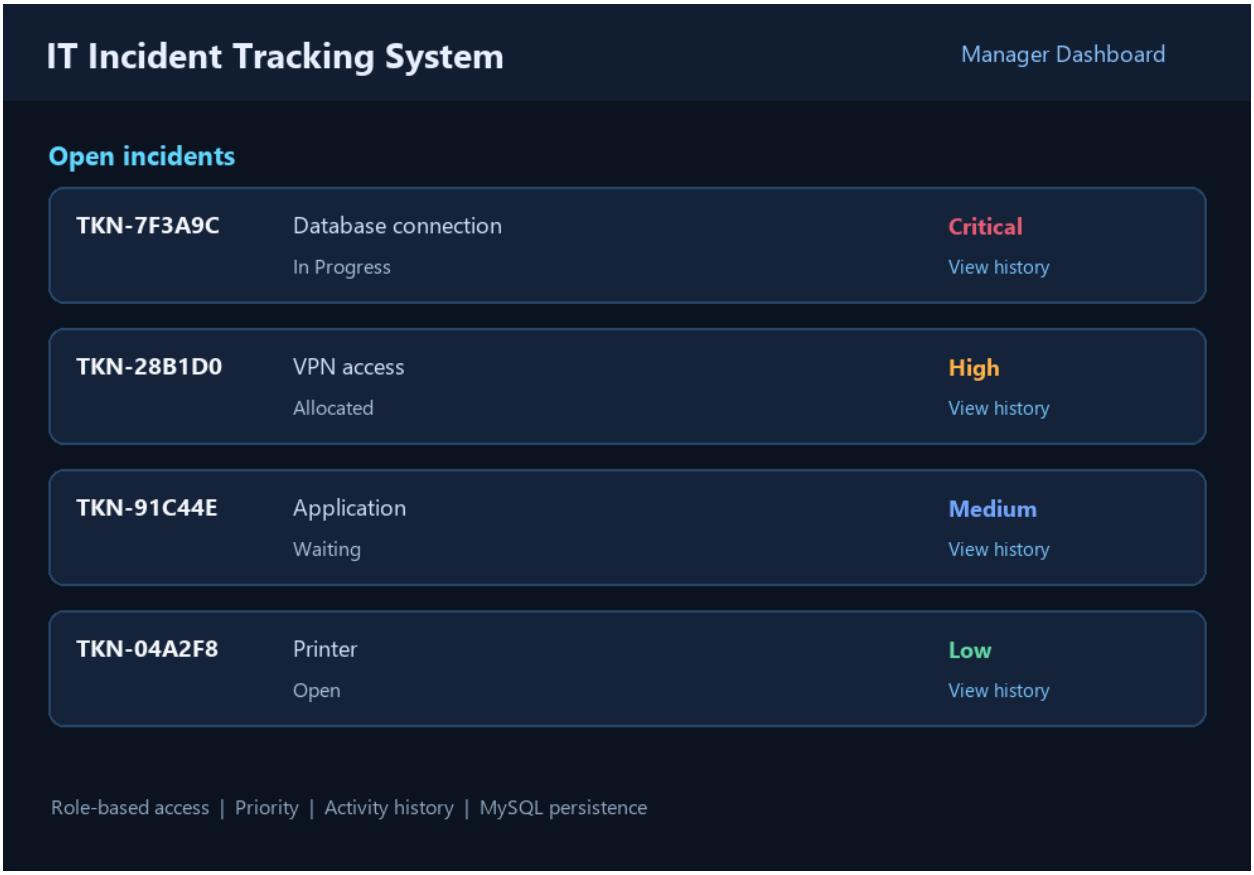
The main design challenge is maintaining consistency between role permissions, ticket status transitions, and activity history. Prepared statements, transaction boundaries, approval states, and role-specific database connections reduce inconsistent or unauthorised updates.

The solution is effective as a concise learning prototype because it connects a familiar service-desk process to database concepts that can be observed directly: keys, constraints, transactions, authentication records, status history, and audit logging.

4. Screen Short

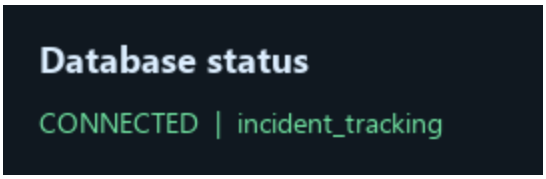
Incident tracking interface and workflow

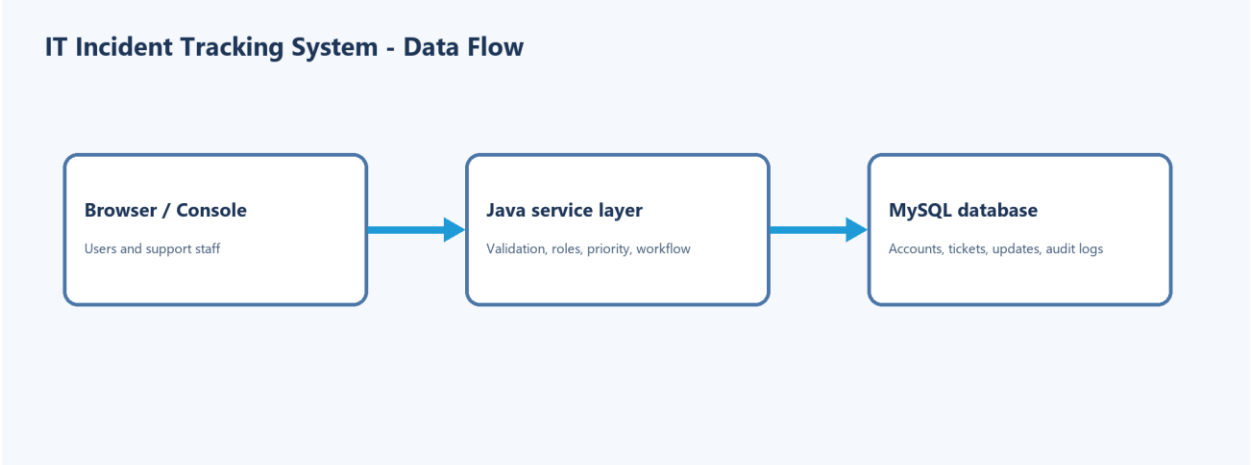
Illustrative project interface: the application presents role-specific incident actions, ticket status, priority, assignment, and history information.



Database status screen - connection check

The application includes a database-status operation that verifies connectivity to the configured MySQL database before normal incident processing.





5. Conclusion

The IT Incident Tracking System demonstrates how a Java application and MySQL database can support a complete incident-management workflow. It records incidents, assigns responsibility, tracks progress, and preserves a history of operational actions.

The database remains the source of truth for users, tickets, statuses, priorities, assignments, and audit events. Validation, prepared statements, password hashing, transactions, and role checks improve reliability and protect important operations.

The project meets its academic objective of applying database management concepts to a practical system. It is a focused prototype rather than a production service desk, and its next stage should concentrate on deployment controls, broader testing, notifications, and reporting.

Overall, the work provides a clear foundation for extending incident tracking into a larger IT support platform while keeping the core data model and workflow understandable.

Future improvements

? Add integration and automated browser tests against a clean disposable MySQL database.

? Add email notifications, SLA timers, escalation rules, attachments, and reporting dashboards.

? Add HTTPS deployment, secret management, backup/recovery procedures, monitoring, and rate limiting.

? Introduce stronger administrative separation, configurable workflows, and detailed analytics.

? Improve accessibility and provide a responsive staff dashboard for operational teams.

6. AI Tool Usage Report (Mandatory)

Purpose of usage

Name of AI tool: AI assistant using Copilot SDK in VS Code. The assistant was used to inspect the existing project structure, understand the Java and MySQL implementation, prepare precise report content, and format the report using the supplied university template.

Project support: The assistant reviewed the README, source packages, database migration, and project configuration to describe authentication, incident processing, roles, ticket status flow, and audit logging without changing the college logo or document styling.

Verification support: The assistant checked the available project artefacts and prepared the final report document and PDF. Claims in this report are limited to functionality visible in the supplied source and database files.

Sample prompts and tasks

Representative task: Inspect the incident-tracking project and supplied report template; replace only the sample written matter and student names; preserve the university format and logo; export the completed report as a PDF.

Outputs and modifications

Outputs included a concise project analysis, updated student and project details, revised report sections and database tables, and a PDF generated from the preserved template layout.

Application source changes: None were required for preparation of this report.

Contribution and responsibility

The AI assistant supported documentation and formatting. The students remain responsible for the project implementation, reviewing the report for academic accuracy, and complying with institutional AI-disclosure requirements.

GITHUB LINK: